

Companion AI — System Briefing

ProjectExtract • Unreal Engine 5.7 • C++ / Behavior Tree

*An AI companion teammate that **perceives enemies, fights them in bursts, follows the player in formation, and revives the player when downed** — built on Unreal's standard AI stack (AIController + Behavior Tree + Blackboard + Perception).*

What the companion does

Behaviour	Trigger	What happens
Formation follow	Default / nothing else active	Walks in a slot 350 back, 200 right of the player. Sprints to catch up if distance exceeds threshold.
Combat	An enemy is sighted	Turns toward target, fires in 1.5 s bursts with a 0.5 s pause, reloads when empty. Inaccuracy 8° at first, tightens to 1.5° over 1.5 s as the companion settles.
Revive	Player goes DBNO	Highest priority — aborts whatever else is running, sprints to the player, holds for 4 s, restores health to 30% and starts shield regen.

Enemy AI is the target dummy: simpler — finds closest non-enemy character and fires within range.

Architecture (the layers)

```
ACompanionAIController ← the "brain": perception + BT runner
|
+-- AIPerceptionComponent (sight 30 m, hearing 20 m)
+-- BlackboardComponent (PlayerActor, CombatTarget, PlayerNeedsRevive, ...)
+-- Runs → BT_Companion (the behavior tree asset)
    |
    +-- Service: UpdateCompanionState (0.25 s heartbeat)
    +-- Decorators: CombatTarget is Set, PlayerNeedsRevive
    +-- Tasks: FollowPlayer, CompanionCombat, RevivePlayer, MoveToCover
```

Four roles to remember:

- **Service** — heartbeat that updates shared state ("who should I fight right now?").
- **Decorator** — conditional gate on a branch ("only run this if CombatTarget is set").
- **Task** — the actual action ("fire a burst", "revive the player").
- **Blackboard** — shared whiteboard the service writes and tasks read.

Key design decisions

Gameplay Tags for identity, not class checks. Enemies carry **Character.Enemy**, companion has **Character.Companion**. The target filter just asks "do you have the enemy tag?" — new enemy types don't need new code.

Data-driven tuning. Ranges, fire rates, inaccuracy, revive duration are all **UPROPERTY(EditAnywhere)** — tuned in Blueprint, not recompiled.

Decoupled body rotation from weapon aim. The body rotates on yaw only (capsules stay upright). The weapon traces directly from muzzle to target location. When a skeletal mesh replaces the cylinder, an aim-offset anim can tilt the arms up/down with no code change.

Authority model. Gameplay state (health, ammo, damage, revive) happens on the server via **HasAuthority()** checks and replicates to clients. Multiplayer-ready while testing in PIE.

Composition over inheritance. Health is a Component bolted onto both player and companion — same damage/death/regen logic, no duplication.

Combat flow (end-to-end walk-through)

1. Enemy spawns. Its constructor adds the **Character.Enemy** gameplay tag.
2. Companion perceives the enemy. **AI Perception** catches it within 30 m sight and a 90° cone.
3. **BTService_UpdateCompanionState** ticks every 0.25 s. Iterates perceived actors → filters by tag → picks closest alive one → writes it to the **CombatTarget** blackboard key.
4. Blackboard decorator fires. "**CombatTarget is Set**" activates the combat branch and aborts lower-priority branches (so follow is interrupted).
5. **BTTask_CompanionCombat** takes over. Checks distance and line-of-sight, rotates body yaw toward the target (smoothly interpolated), calls **StartWeaponFire()** in bursts.
6. **AWeaponBase::PerformHitscan** runs. Detects AI owner, looks up the companion's aim target, traces from muzzle → target location with a small random spread, applies damage.
7. Enemy dies → service clears **CombatTarget**. Decorator drops the branch, BT falls back to follow-player.
8. Player goes DBNO mid-fight → revive branch. **PlayerNeedsRevive** flips true, the revive decorator *aborts both* (even combat), companion sprints to revive, holds 4 s, restores health, restarts shield regen.

Working today

- ✓ Perception, target acquisition, gameplay-tag filtering
- ✓ Combat: rotate, burst-fire, reload, inaccuracy settling
- ✓ Weapon aim: muzzle → target location (handles ramps, height differences)
- ✓ Revive: DBNO detection → sprint → hold → restore → shield regen resumes
- ✓ Branch priorities: revive interrupts combat, combat interrupts follow
- ✓ Diagnostic log category **LogCompanionAI** + debug-draw lines (green / red / yellow)

Known trade-offs / next steps

Showing awareness of what isn't done and why:

- **Cover is disabled.** **MoveToCover** task works, but the current BT uses a **Sequence** instead of a **Simple Parallel**, so combat holds the branch and cover never runs alongside. Fix: swap to **Simple Parallel**.
- **No move-to-engage.** If the enemy is sighted but out of range, the combat task fails instead of closing distance. Trivial fix — call **MoveToActor** in the out-of-range branch.
- **LOS trace doesn't ignore the player.** The player's body can block the companion's line of fire.
- **Weapon has no muzzle socket.** Shots originate from the weapon root. Cosmetic — fixed by giving the assault rifle a skeletal mesh with a **Muzzle** socket.

What I learned

- How UE5's BT + Blackboard + Perception compose cleanly — each piece has one job.
- How to diagnose AI behaviour by instrumenting decision gates (**bDebugLogging** uproperty + the built-in BT debugger tool).
- When to use **Sequence** vs **Selector** vs **Simple Parallel** (the cover bug was a concrete lesson).
- How to decouple character rotation from weapon aim so gameplay code is future-proof against rigged meshes.